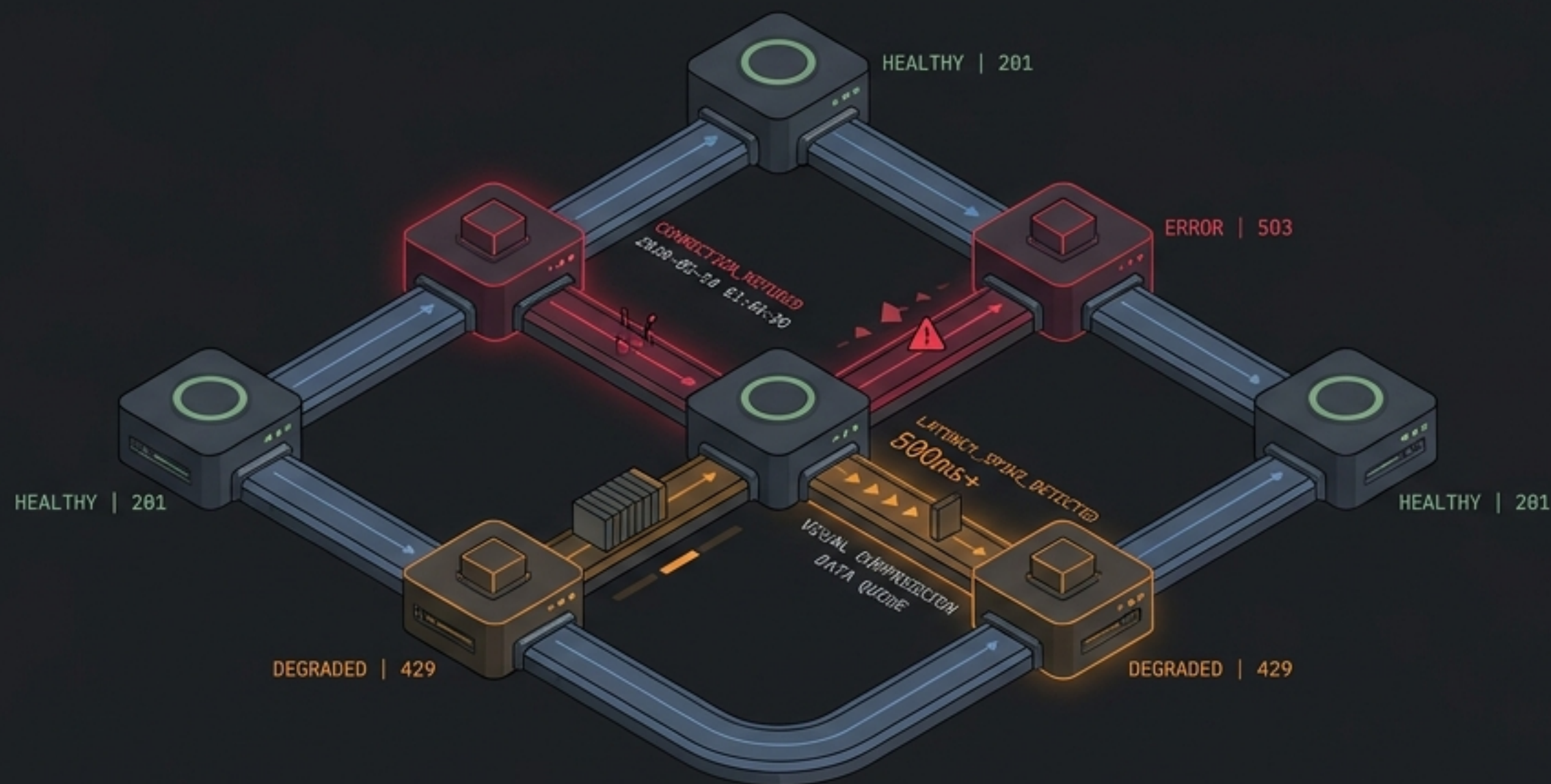




Agentic SRE Testbed

Руководство по эксплуатации и симуляции инцидентов

Идеология и 3 плоскости стенда

Управляющая плоскость (Control Plane)

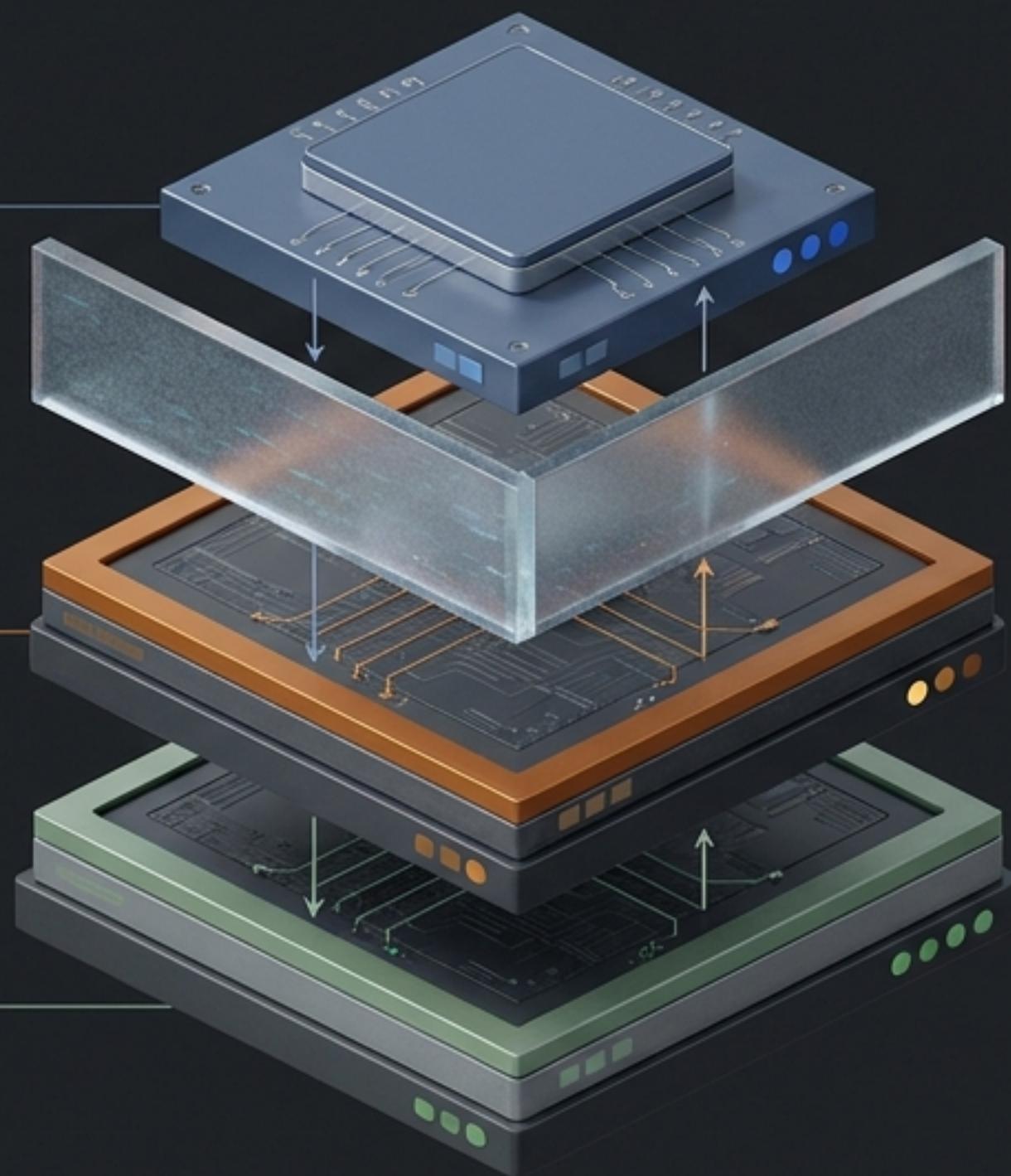
fault-injector,
experiment-runner, portal

Диагностическая плоскость

Prometheus, Loki, Tempo,
Grafana

Плоскость приложения

nginx, gateway, auth, order,
payment, PostgreSQL, Redis



Главное правило системы:
Управляющая плоскость
абсолютно невидима для
диагностической.

- Логи Control Plane не попадают в Loki. Метрики не видны в Prometheus.
- Агент (и вы) видит только симптомы, а не причину поломки. Расследование происходит «вслепую», как в реальном продакшене.

Требования и быстрый старт

Системные требования

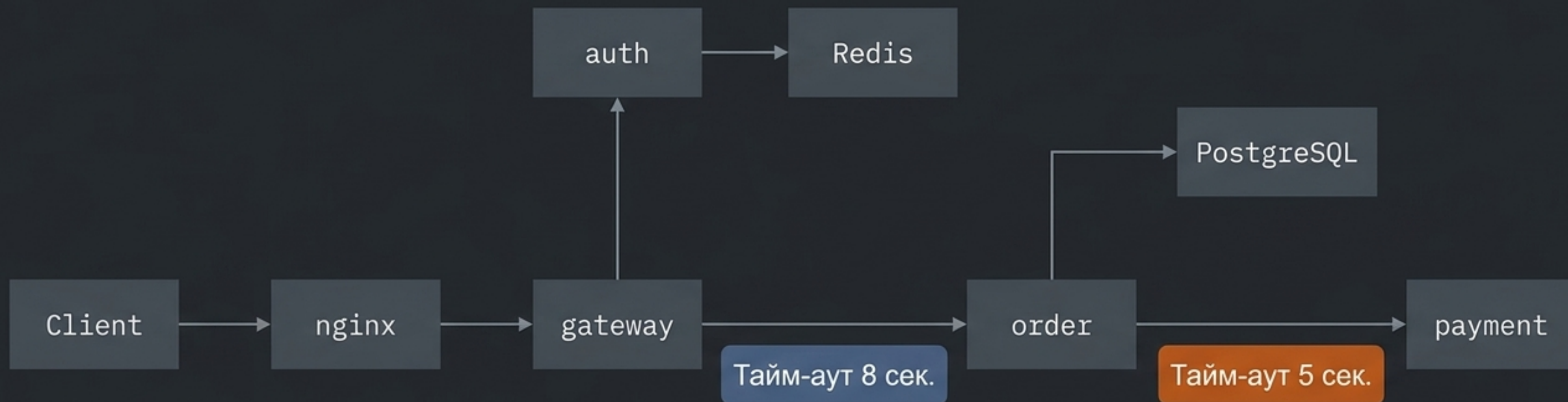
✓	Docker Desktop / Compose v2 4 CPU, 6 ГБ RAM, 18 контейнеров
✓	GNU Make make --version
✓	curl 7.76+
✓	python3

3 минуты до инцидента

```
x ● ● _ □ x
> make up
(Сборка и запуск)
> make smoke
(Проверка за 5 сек)
> make open
(Открывает http://localhost:8000)

ok  app: ready, login, order 201
ok  no active faults
ok  loki reachable
ok  portal http://localhost:8000: every tool
    reachable
```

Архитектура: 6 микросервисов



Вложенные тайм-ауты гарантируют, что order всегда успевает зафиксировать payment_failed до того, как gateway сбросит соединение по тайм-ауту.

API приложения и Матрица статусов

Auth (POST /login)

alice/alice -> Returns Bearer Token

Order (POST /orders)

```
{"item": "book", "quantity": 1, "amount_cents": 1500}
```

201

Успешно (paid).

500

Внутренняя ошибка (БД отказала в соединении).

502

Зависимость упала (платеж не прошел, заказ: payment_failed).

503

БД не ответила за 10с или сбой записи.

504

Тайм-аут Gateway (ждал > 8с).

Анатомия трейсинга: Идентификаторы расследования



X-Request-ID

Инжектируется на уровне nginx. Присутствует в каждой строке лога каждого сервиса.

Можно передать свой кастомный ID:

-H 'X-Request-ID: demo-42'

X-Trace-ID

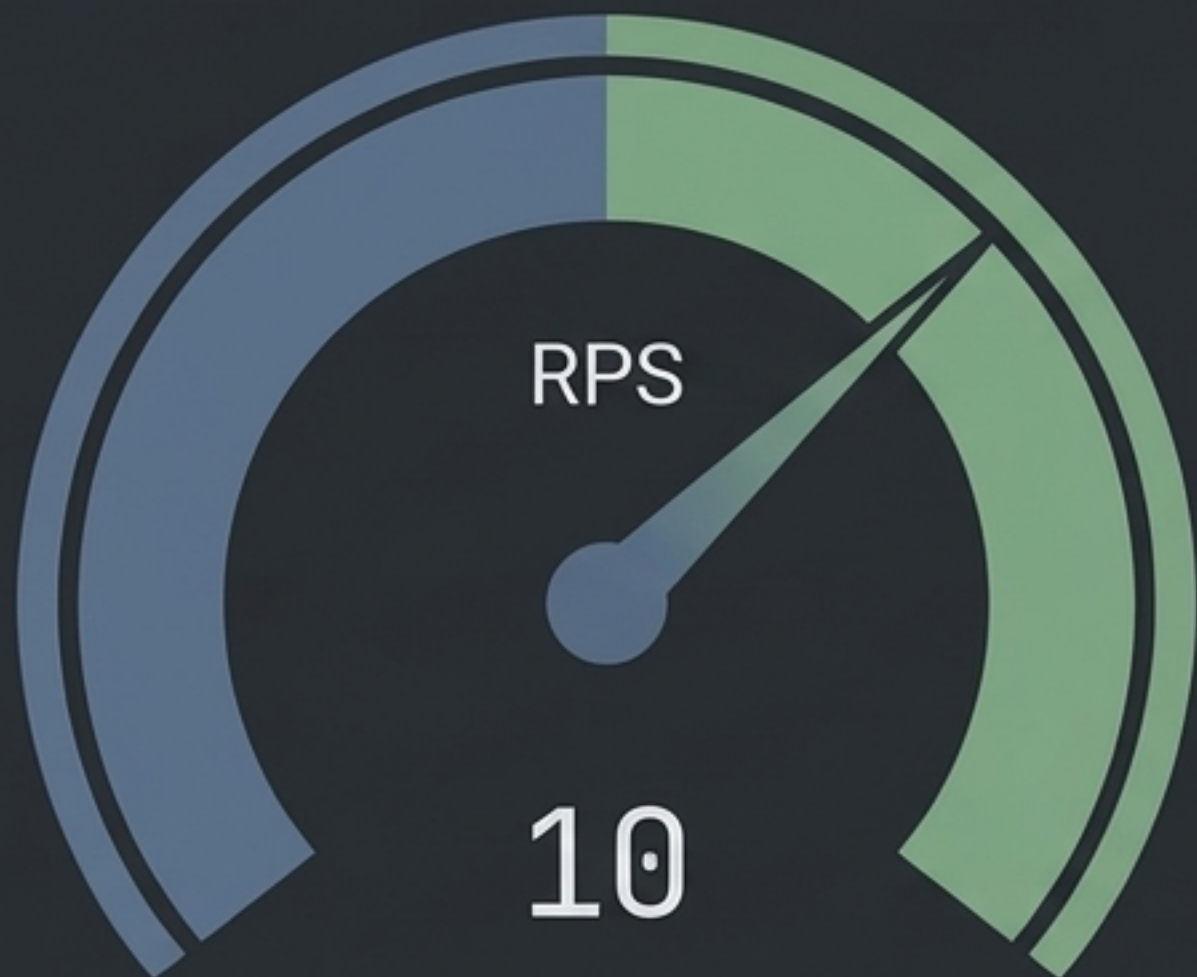
Генерируется для Tempo.

Связывает логи со спанами трейсинга.

100% сэмплинг запросов.

Pro-tip: В Control Center клик на «trace ↗» рядом со строкой лога мгновенно открывает водопад спанов.

Генерация фонового шума и нагрузки



Чтобы расследовать инцидент, нужен поток данных. Нагрузка имитирует реального клиента (создание заказов, чтение, редкие ошибки логина).

```
> make load  
(5 RPS на 60 сек)
```

```
> make load RATE=10 DURATION=300  
(10 RPS на 5 мин)
```

```
> make load RATE=5 DURATION=600 &  
(Запуск в фоне, пока вы ломаете систему)
```

Телеметрия: Инструментарий SRE

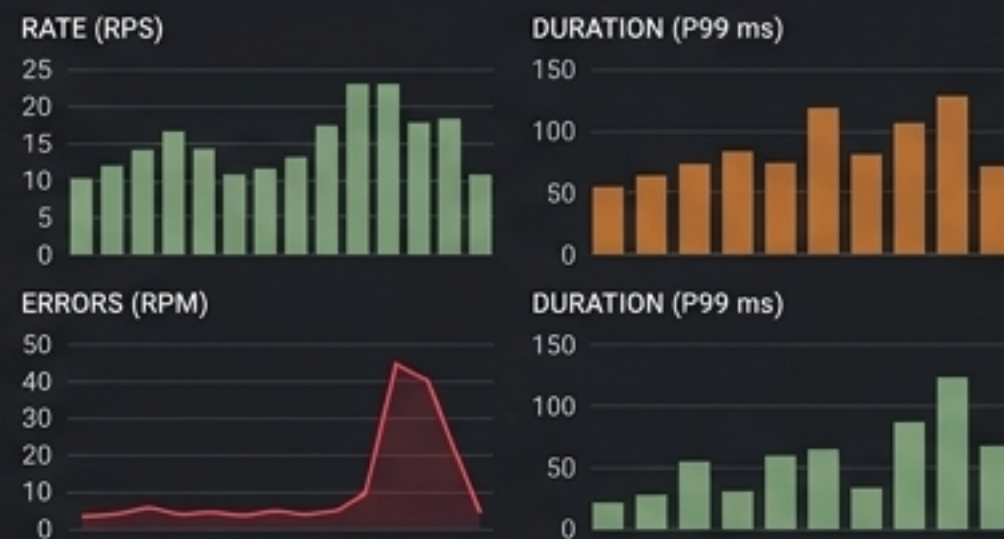
Единый интерфейс Control Center

<http://localhost:8000>
консолидирует все инструменты. Данные хранятся 48 часов.

Investigation Mode

Скрывает подсказки и истинные причины, оставляя только голые симптомы для расследования.

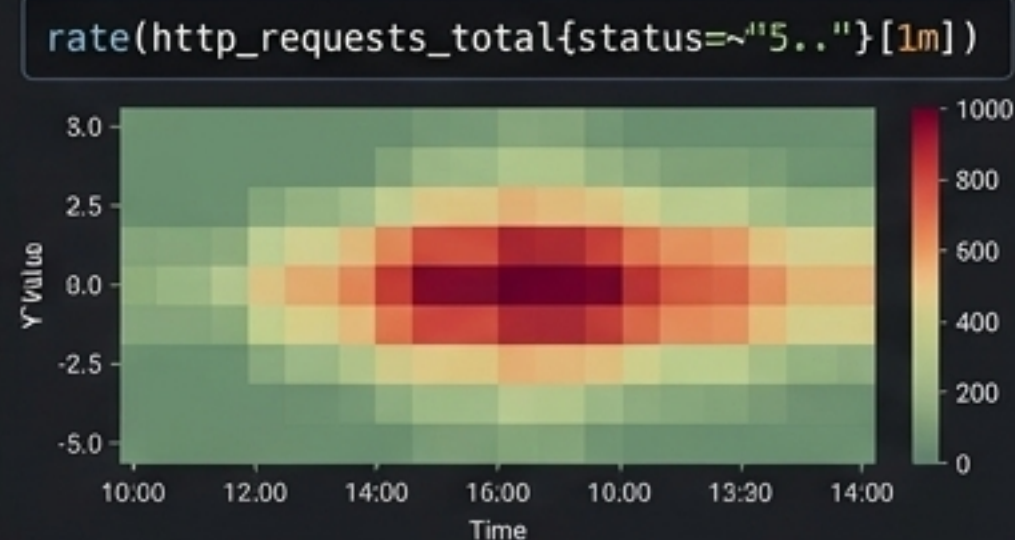
Grafana



Loki

```
{"timestamp":"2024-10-27T10:01:00Z","level":"INFO","service":"payment","msg":"Processing order","order_id":"12345"}
{"timestamp":"2024-10-27T10:01:01Z","level":"ERROR","service":"payment","msg":"Database connection failed","error_code":"503"}
{"timestamp":"2024-10-27T10:01:02Z","level":"WARN","service":"inventory","msg":"Low stock detected","item_id":"98765"}
{"timestamp":"2024-10-27T10:01:02Z","level":"WARN","service":"inquiry","msg":"Low stock detected"}
```

Prometheus



Tempo



5 дашбордов Grafana

Название	Данные	Диагностический вопрос
Service Overview	RED метрики, латентность, CPU/RAM/PSI	Что болит?
Dependencies	Графы вызовов (gateway→auth, order→postgres), исходы, блокировки БД	Какая зависимость тормозит или падает?
Logs	Логи с фильтрами (service, level, request_id)	Что произошло с конкретным запросом?
Traces	Медленные трейсы POST /orders, метрики рёбер	Где именно теряется время?
Diagnostic Domains	Сигналы по доменам (app, db, redis, payment)	Какой домен деградировал?

🔒 *Внимание: домен 'проху' на дашборде Diagnostic Domains намеренно скрыт для сложных сценариев расследования.*

Язык запросов: LogQL и PromQL

LogQL (Loki) - Поиск иголки в стоге сена

```
1 {service=~".+"} |= "demo-42"           // Весь путь запроса по request_id
2 {service="order"} | json | level="error" // Ошибки сервиса order
3 {service="nginx"} | json | request_time_s > 1 // Медленные запросы на входе
```

PromQL (Prometheus) - Анализ трендов

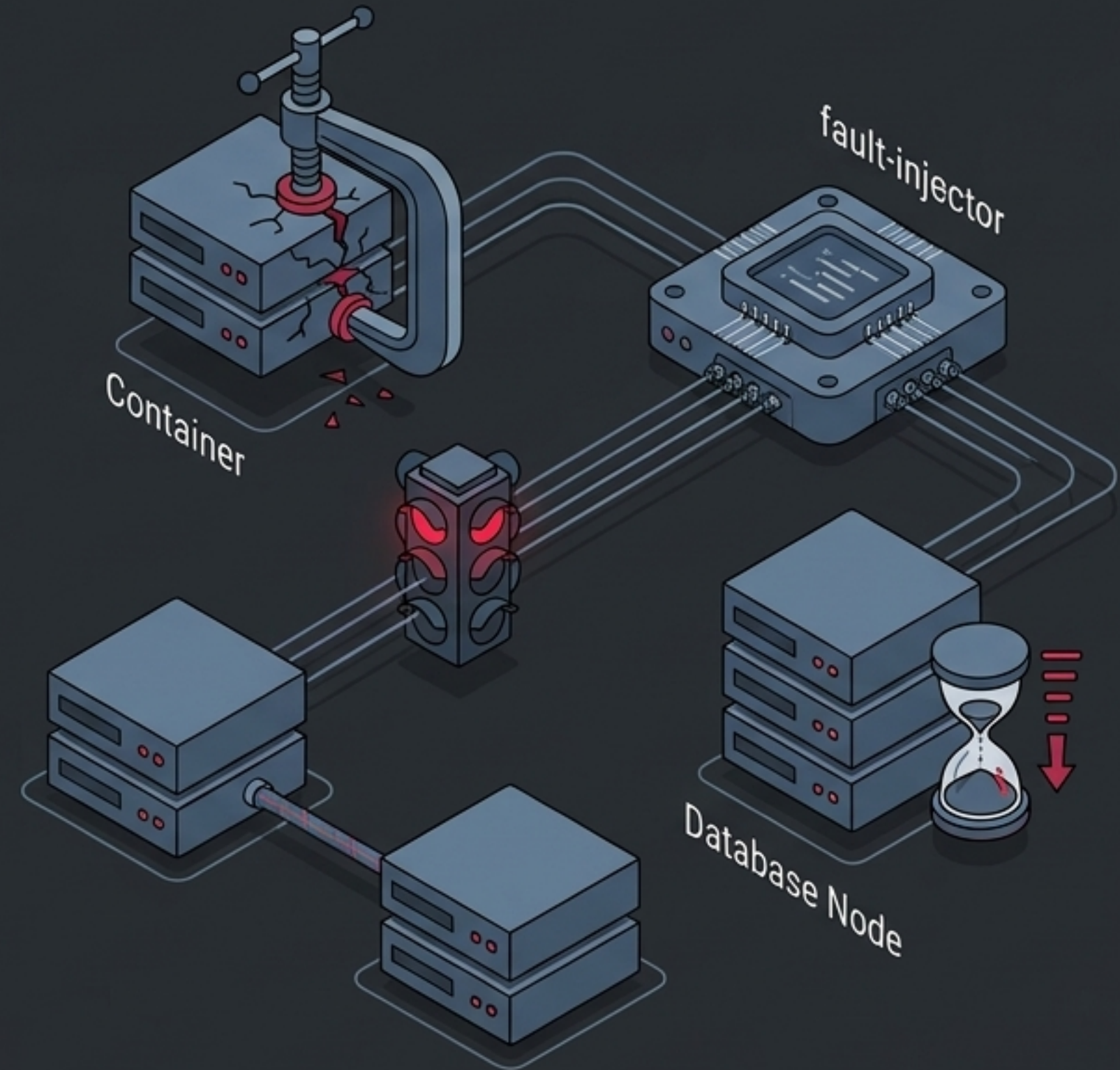
```
1 histogram_quantile(0.95, sum by (le)
  (rate(http_request_duration_seconds_bucket{job="gateway",route="/orders"}[1m])))
  // p95 создания заказа
2 sum by (job, dependency, outcome)
  (rate(dependency_requests_total{outcome!="success"}[1m]))
  // Неудачные вызовы зависимостей
```

Механика разрушения (Фолты)

Механизмы разрушения настоящие: остановка контейнеров, tc/iptables в сети, триггеры pg_sleep, cgroups для ресурсов.

Фолты пишутся в SQLite и переживают рестарты.

```
> make fault TYPE=payment-latency  
PARAMS='{\"latency_ms\":2500}'  
> make fault TYPE=cpu-saturation  
TARGET=order  
> make faults (список активных)  
> make recover (снять всё)
```



Каталог уязвимостей (Часть 1)

Приложение (App)		
Тип	Цель	Механика
payment-latency / payment-error	payment	Имитация задержек и 5xx
intermittent-errors	auth, order, payment	Часть запросов падает
service-unavailable / dependency-timeout	auth, order, payment	Connection refused или пауза контейнера
Ресурсы (Resources)		
cpu-saturation / cpu-limit	любой	Процессы-печки съедают CPU / квота понижена
memory-pressure	любой	Захват памяти внутри лимита 256 МБ
Базы Данных (Databases)		
db-slow-query	postgres	Триггер pg_sleep на запись
db-lock-contention	postgres	Периодический LOCK TABLE orders IN SHARE MODE

Каталог уязвимостей (Часть 2)

Сеть (Network)	
Тип	Механика
network-latency / packet-loss	Задержки tc netem на исходящем трафике, бьет по хвосту латентности
connection-failure	iptables reject (быстрый отказ) или drop (тайм-аут)
Конфигурация и Деплой (Config/Deploy) - Вызывает пересоздание	
Тип	Механика
incorrect-endpoint	Неверный URL зависимости (опечатка)
bad-configuration	Неудачные переменные окружения (например, короткий пул БД)
bad-deployment	Новая версия падает в цикле, тормозит или выдает 5xx
Прокси (Proxy)	
proxy-rate-limit	Строгий limit_req в Nginx (очередь, 503)
proxy-bandwidth-limit	limit_rate (ошибка размерности: медленно, без ошибок)

Матрица симптомов и причин (Triage Matrix)

Симптомы (Symptoms)

Причины (Root Causes)

Медленно, но 201 ОК

payment-latency, db-slow-query, db-lock-contention, cpu-saturation, network-latency, bad-deployment slow, проблемы прокси

Ошибки 5xx

payment-error, db-connection-exhaustion, dependency-timeout, connection-failure, bad-deployment crash

Ошибка 401

bad-configuration (короткий TTL токена)

Снаружи всё нормально

memory-pressure, bad-configuration с маленьким пулом (пока БД не замедлится окончательно)

Жизненный цикл эксперимента



Замер эталона
под нагрузкой.

Включение
фолтов сценария.

Наблюдение
за инцидентом
(p50/p95 spike).

Подсчет
статистики
и сигналов.

Снятие фолтов
(make recover).

Ожидание окна
без симптомов.

```
> make experiment SCENARIO=payment-latency  
(Вердикт покажет expected_seen и recovered)
```

Сценарии инцидентов: Простые и M8 Complex

Простые сценарии (Одиночные отказы)

- payment-latency → Симптом: slow
- redis-unavailable → Симптом: errors
- connection-failure → Симптом: errors

M8 Complex Scenarios (Взлом моделей)

Category 1: Одинаковый симптом slow-payment, slow-db-query, slow-cpu. У всех p95 > 2с и 201 ОК. Отличаются только микро-улики.

Category 2: Ложная корреляция deploy-then-payment. Безвредный релиз order, через 5 минут тормозит payment. Откат релиза не помогает.

Category 3: Ломает модель slow-proxy-network. Все Diagnostic Domains горят зеленым. Проблема скрыта в nginx, видна только в его логах и трейсах.

Тестирование и Приёмка



Приёмка / Acceptance Pipeline

Прогоняет все сценарии подряд: проверяет появление симптома, совпадение картины доменов и успешное восстановление системы.

```
> make acceptance  
(~30 мин, укороченные фазы)  
  
> make acceptance FULL=1 ROUNDS=2  
(~1 ч, полные фазы, проверка детерминированности)
```

Уровни тестирования

```
> make test-faults  
Изолированная проверка инжекторов (~7 мин)  
  
> make test-experiments  
Проверка простых сценариев (~13 мин)  
  
> make test-challenges  
Проверка сложных M8 сценариев (~10 мин)
```

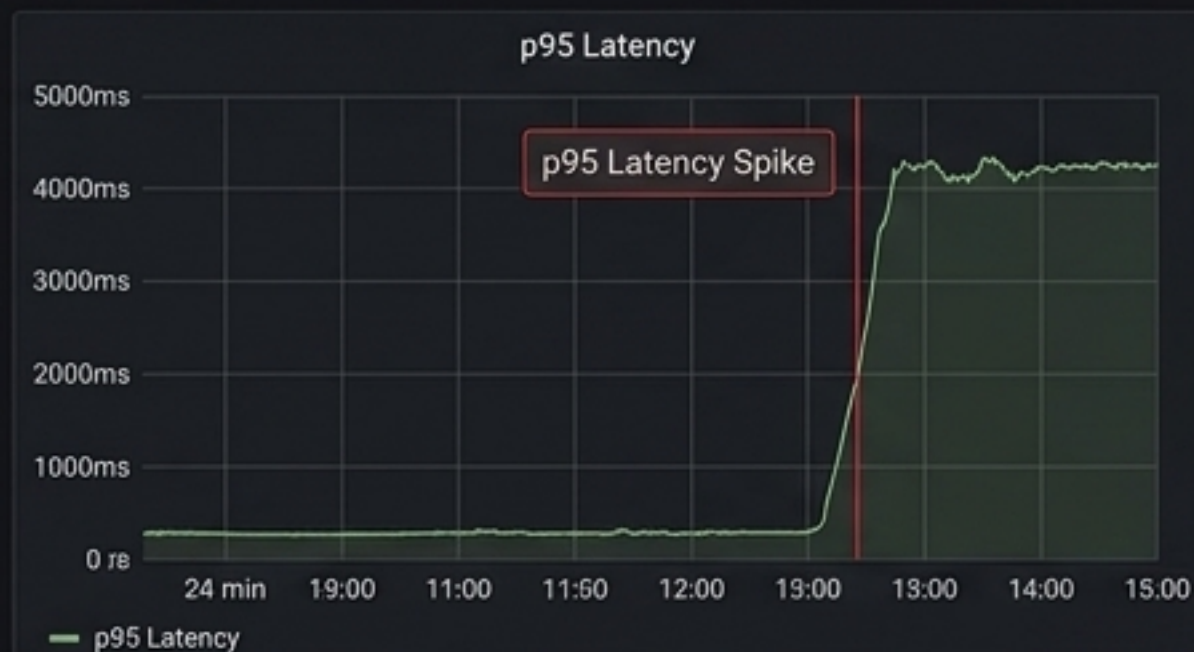
Расследование (Часть 1): СИМПТОМЫ

Command Log

```
> 1. Старт: make load RATE=5  
  
> 2. Инцидент вслепую: make  
fault TYPE=db-lock-contention
```

Overview

Overview



201 OK

Diagnostic Domains



NORMAL

application



DEGRADED

database



NORMAL

redis



NORMAL

payment

Вывод:

Метрика p95 резко взлетает, но статус-коды остаются 201 OK. Это 'slow' группа фолтов. Домен database загорается оранжевым.

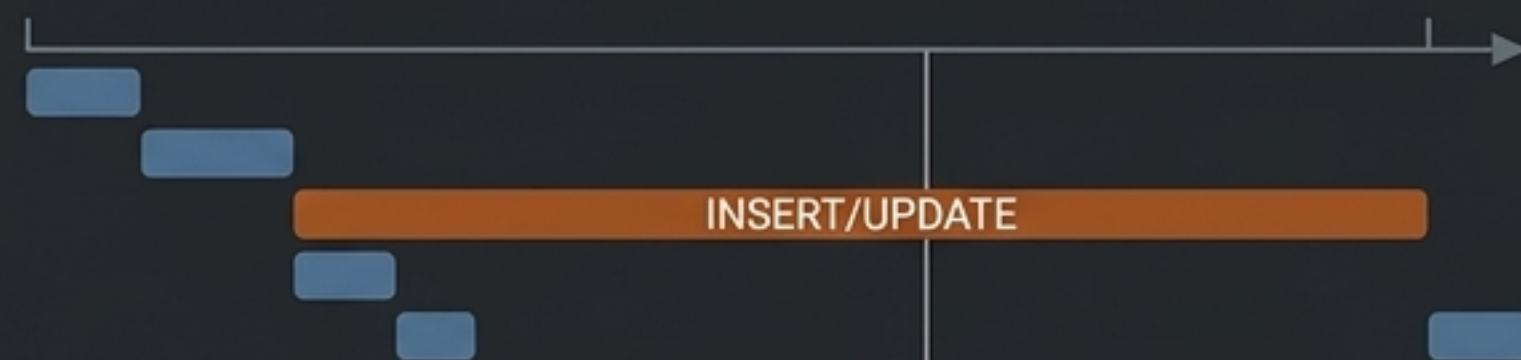
Расследование (Часть 2): Улики и Диагноз

Dependencies Map



Всплески ShareLock в PostgreSQL.

Tempo Traces



Водопад спанов показывает узкое горлышко.

Loki Logs



Ошибок нет. Задержка не ровная, а идет периодическими всплесками (1.5 сек из каждых 2 сек).

Диагноз: `db-lock-contention` на таблице `orders`.

Разрешение: сверка через `make faults` → уборка через `make recover`.

Операции: Конфигурация, Очистка, Устранение проблем

Под капотом (.env)

```
PORTAL_PORT=8000  
NGINX_PORT=8080  
GATEWAY_HTTP_TIMEOUT_SECONDS=8  
DB_POOL_MAX_SIZE=30
```

Настройки читают и Compose, и Makefile.

Очистка и Сброс

make recover
(Мягко: снимает фолты,
данные целы)

make down
(Пауза: стоп контейнеров,
тома целы)

make reset
(Ядерная опция: удаляет ВСЁ
включая БД → запуск с нуля)

Типичные проблемы

Q: Ошибка 409?

→ A: Конфликт фолтов
(make recover)

Q: Мас уснул на тестах?

→ A: Запускай через
caffeinate -i

Q: 502 после редеплоя?

→ A: Нормально (DNS
кэш), подождите пару
секунд

Шпаргалка: Команды Makefile

Жизненный цикл (Lifecycle)

```
make up / make down / make reset  
make smoke / make ps / make logs
```

Нагрузка и Телеметрия (Load & Telemetry)

```
make open (http://localhost:8000)  
make load RATE=5 DURATION=120
```

Хаос и Инциденты (Chaos)

```
make fault TYPE=<тип> TARGET=<цель>  
make faults (Активные) / make recover (Очистить)
```

Эксперименты и Тесты (Experiments)

```
make scenarios / make experiment SCENARIO=<имя>  
make acceptance FULL=1 / make test
```

The system is broken. Go fix it. → <http://localhost:8000>